Name: Prasana Shangloo
Roll No.: 23102A0076
Branch: CMPN A

```
import io, zipfile, warnings, requests
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.pipeline import make_pipeline
from sklearn.inspection import permutation_importance
from sklearn.metrics import (silhouette_score, adjusted_rand_score, accuracy_score, precision_score,
                             recall_score, f1_score, roc_auc_score, confusion_matrix,
                             ConfusionMatrixDisplay, roc_curve)

warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
RANDOM_STATE = 42
```

```
URL = "https://archive.ics.uci.edu/static/public/352/online+retail.zip"
try:
    r = requests.get(URL, timeout=180)
    r.raise_for_status()
    z = zipfile.ZipFile(io.BytesIO(r.content))
    xlsx_name = [n for n in z.namelist() if n.lower().endswith(".xlsx")][0]
    df_raw = pd.read_excel(z.open(xlsx_name))          # takes ~1-2 minutes
except Exception as e:
    print("Automatic download failed:", e)
    print("Download 'Online Retail.xlsx' from the UCI page and upload it below.")
    from google.colab import files
    uploaded = files.upload()
    df_raw = pd.read_excel(next(iter(uploaded)))

print(df_raw.shape)
df_raw.head()
```

```
(541909, 8)
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	17850.0	United Kingdom
			KNITTED UNION FLAG HOT WATER		2010-12-01			United

```
print("Missing values per column:")
print(df_raw.isna().sum(), "\n")

df = df_raw.copy()
n0 = len(df)

df = df.dropna(subset=["CustomerID"])
print(f"After dropping missing CustomerID : {len(df):,} (removed {n0 - len(df):,})")

is_cancel = df["InvoiceNo"].astype(str).str.startswith("C")
print(f"Cancellation rows removed         : {is_cancel.sum():,}")
df = df[~is_cancel]

df = df[(df["Quantity"] > 0) & (df["UnitPrice"] > 0)]
print(f"After removing invalid qty/price  : {len(df):,}")
```

```
before = len(df)
df = df.drop_duplicates()
print(f"Duplicates removed               : {before - len(df):,}")

df["CustomerID"] = df["CustomerID"].astype(int)
df["InvoiceDate"] = pd.to_datetime(df["InvoiceDate"])
df["TotalPrice"] = df["Quantity"] * df["UnitPrice"]
print(f"\nFinal transactions: {len(df):,} | Customers: {df['CustomerID'].nunique():,}")
```

```
Missing values per column:
InvoiceNo           0
StockCode           0
Description      1454
Quantity            0
InvoiceDate         0
UnitPrice           0
CustomerID     135080
Country             0
dtype: int64

After dropping missing CustomerID : 406,829 (removed 135,080)
Cancellation rows removed         : 8,905
After removing invalid qty/price  : 397,884
Duplicates removed                : 5,192

Final transactions: 392,692 | Customers: 4,338
```

```
snapshot = df["InvoiceDate"].max() + pd.Timedelta(days=1)
g = df.groupby("CustomerID")

cust = pd.DataFrame({
    "Recency": (snapshot - g["InvoiceDate"].max()).dt.days,
    "Frequency": g["InvoiceNo"].nunique(),
    "Monetary": g["TotalPrice"].sum(),
    "TotalQuantity": g["Quantity"].sum(),
    "UniqueProducts": g["StockCode"].nunique(),
    "FirstPurchase": g["InvoiceDate"].min(),
})
cust["AvgTransactionValue"] = cust["Monetary"] / cust["Frequency"]
cust["AvgQuantityPerOrder"] = cust["TotalQuantity"] / cust["Frequency"]
tenure_months = ((snapshot - cust["FirstPurchase"]).dt.days / 30).clip(lower=1)
cust["PurchaseFrequency"] = cust["Frequency"] / tenure_months
cust = cust.drop(columns="FirstPurchase")

features = ["Recency", "Frequency", "Monetary", "AvgTransactionValue",
            "TotalQuantity", "AvgQuantityPerOrder", "PurchaseFrequency", "UniqueProducts"]
cust = cust[features]
print(cust.shape)
cust.describe().round(2)
```

(4338, 8)

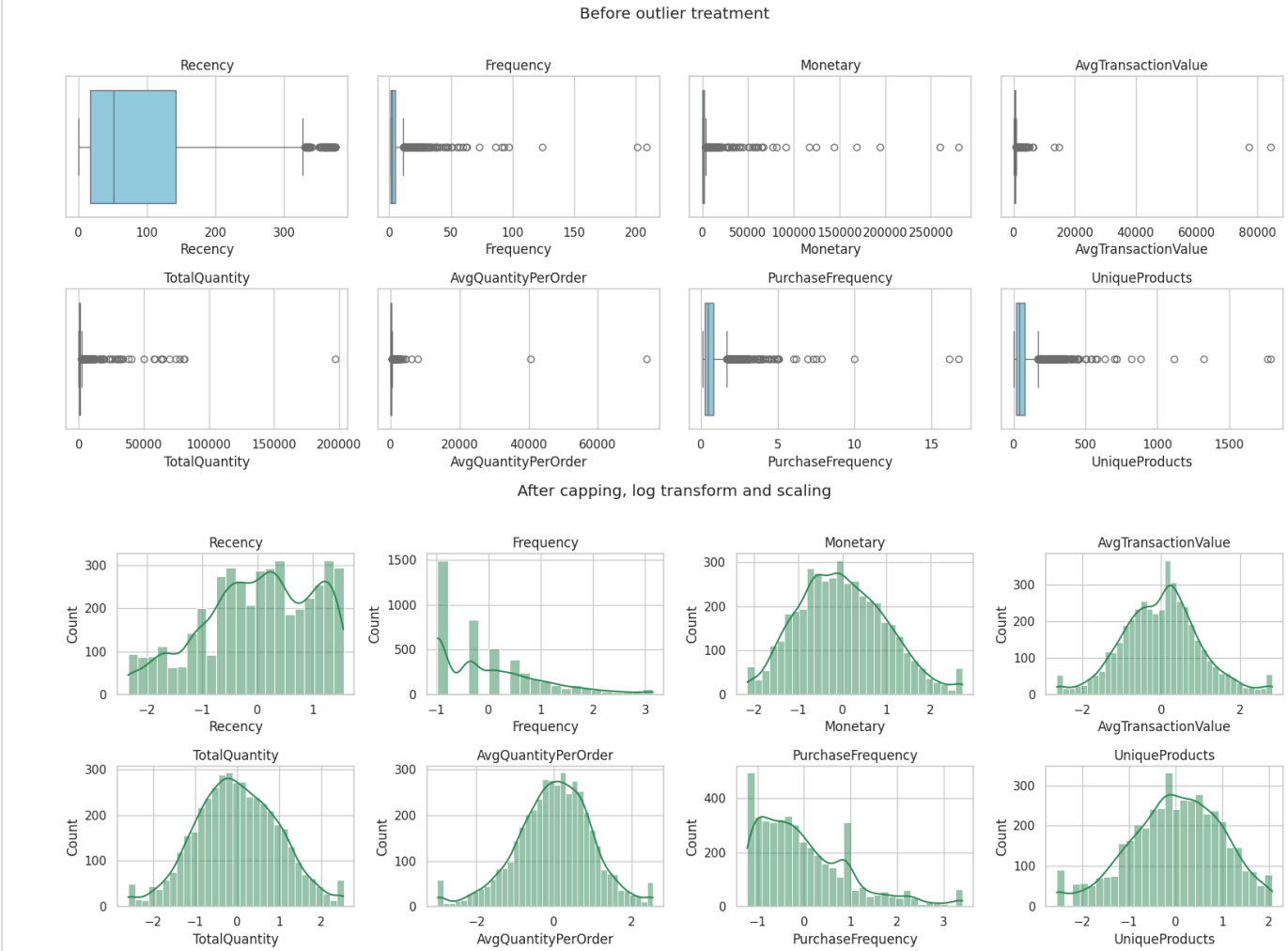
	Recency	Frequency	Monetary	AvgTransactionValue	TotalQuantity	AvgQuantityPerOrder	PurchaseFrequency	UniqueProducts
count	4338.00	4338.00	4338.00	4338.00	4338.00	4338.00	4338.00	4338.00
mean	92.54	4.27	2048.69	417.65	1187.64	252.46	0.64	61.50
std	100.01	7.70	8985.23	1796.51	5043.62	1312.77	0.74	85.37
min	1.00	1.00	3.75	3.45	1.00	1.00	0.08	1.00
25%	18.00	1.00	306.48	177.87	159.00	92.00	0.23	16.00
50%	51.00	2.00	668.57	291.94	378.00	161.00	0.45	35.00
75%	142.00	5.00	1660.60	428.28	989.75	270.00	0.81	77.00
max	374.00	209.00	280206.02	84236.25	196915.00	74215.00	16.76	1787.00

```
fig, axes = plt.subplots(2, 4, figsize=(16, 6))
for ax, col in zip(axes.ravel(), features):
    sns.boxplot(x=cust[col], ax=ax, color="skyblue")
    ax.set_title(col)
plt.suptitle("Before outlier treatment", y=1.02); plt.tight_layout(); plt.show()

lo, hi = cust[features].quantile(0.01), cust[features].quantile(0.99)
capped = cust[features].clip(lower=lo, upper=hi, axis=1)
logged = np.log1p(capped)
```

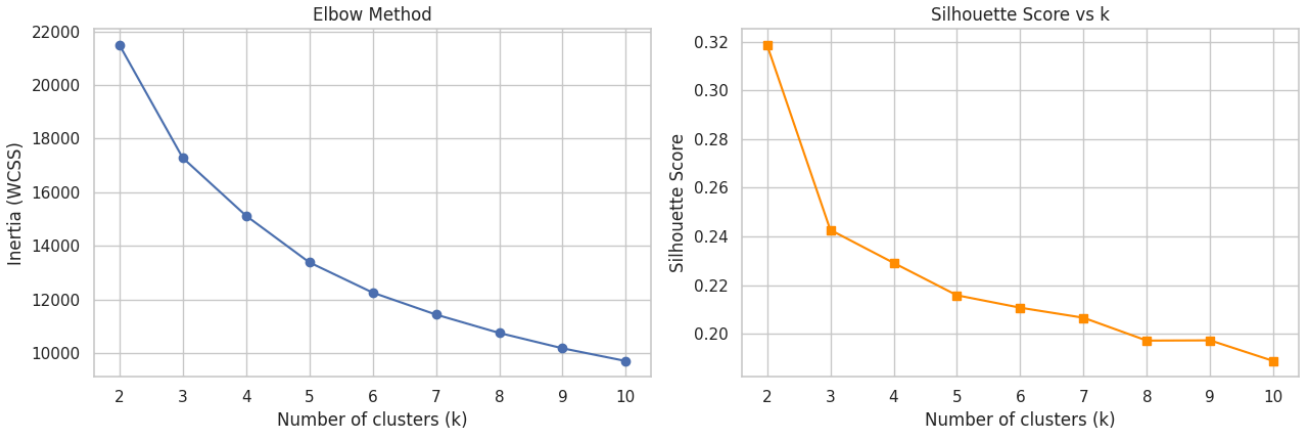
```
scaler = StandardScaler()
X = scaler.fit_transform(logged)
X_df = pd.DataFrame(X, columns=features, index=cust.index)

fig, axes = plt.subplots(2, 4, figsize=(16, 6))
for ax, col in zip(axes.ravel(), features):
    sns.histplot(X_df[col], kde=True, ax=ax, color="seagreen")
    ax.set_title(col)
plt.suptitle("After capping, log transform and scaling", y=1.02); plt.tight_layout(); plt.show()
```



```
ks = range(2, 11)
inertia, sil = [], []
for k in ks:
    km = KMeans(n_clusters=k, n_init=10, random_state=RANDOM_STATE).fit(X)
    inertia.append(km.inertia_)
    sil.append(silhouette_score(X, km.labels_, sample_size=min(5000, len(X)), random_state=RANDOM_STATE))

fig, ax = plt.subplots(1, 2, figsize=(13, 4.5))
ax[0].plot(list(ks), inertia, "o-"); ax[0].set_title("Elbow Method")
ax[0].set_xlabel("Number of clusters (k)"); ax[0].set_ylabel("Inertia (WCSS)")
ax[1].plot(list(ks), sil, "s-", color="darkorange"); ax[1].set_title("Silhouette Score vs k")
ax[1].set_xlabel("Number of clusters (k)"); ax[1].set_ylabel("Silhouette Score")
plt.tight_layout(); plt.show()
print(pd.DataFrame({"k": list(ks), "Inertia": np.round(inertia, 1), "Silhouette": np.round(sil, 4)}))
```




```
    k  Inertia  Silhouette
0   2  21502.5      0.3187
1   3  17286.0      0.2427
2   4  15126.2      0.2291
3   5  13384.7      0.2158
4   6  12256.5      0.2107
5   7  11436.2      0.2066
6   8  10749.0      0.1972
7   9  10175.7      0.1973
8  10   9700.8      0.1888
```

```
K = 4   # <-- set from the Elbow / Silhouette plots above

kmeans = KMeans(n_clusters=K, n_init=20, random_state=RANDOM_STATE).fit(X)
cust["Cluster"] = kmeans.labels_
sil_km = silhouette_score(X, kmeans.labels_)
print(f"K-Means (k={K}) Silhouette Score: {sil_km:.4f}")
print(cust["Cluster"].value_counts().sort_index())
```

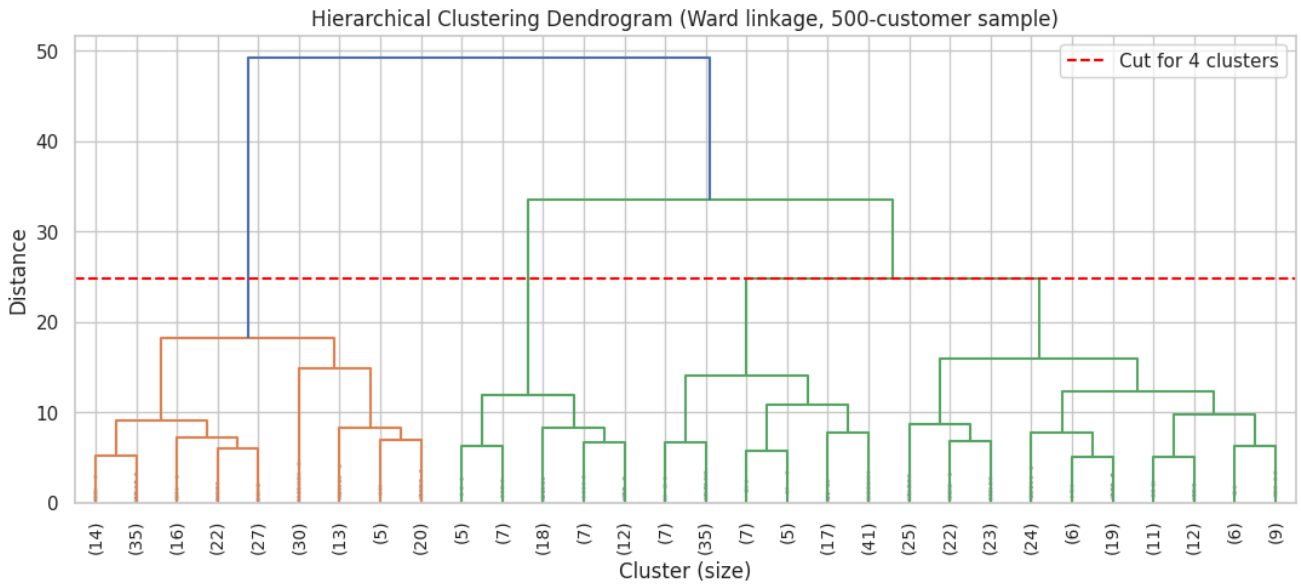
```
K-Means (k=4) Silhouette Score: 0.2291
Cluster
0    1132
1    1063
2     933
3    1210
Name: count, dtype: int64
```

```
# Dendrogram on a random sample (a full dendrogram of thousands of customers is unreadable)
rng = np.random.RandomState(RANDOM_STATE)
idx = rng.choice(len(X), size=min(500, len(X)), replace=False)
Z = linkage(X[idx], method="ward")

plt.figure(figsize=(13, 5))
dendrogram(Z, truncate_mode="lastp", p=30, leaf_rotation=90, show_contracted=True)
plt.axhline(y=Z[-(K - 1), 2] - 1e-6, color="red", linestyle="--", label=f"Cut for {K} clusters")
plt.title("Hierarchical Clustering Dendrogram (Ward linkage, 500-customer sample)")
plt.xlabel("Cluster (size)"); plt.ylabel("Distance"); plt.legend(); plt.show()

# Full-data agglomerative clustering with the same k
```

```
agg = AgglomerativeClustering(n_clusters=K, linkage="ward").fit(X)
cust["HCluster"] = agg.labels_
sil_hc = silhouette_score(X, agg.labels_)
print(f"Hierarchical (k={K}) Silhouette Score: {sil_hc:.4f}")
```




```
Hierarchical (k=4) Silhouette Score: 0.1958
```

```
ari = adjusted_rand_score(cust["Cluster"], cust["HCluster"])
comparison = pd.DataFrame({
    "Method": ["K-Means", "Hierarchical (Ward)"],
    "Clusters": [K, K],
    "Silhouette Score": [sil_km, sil_hc],
})
display(comparison.round(4))
print(f"Adjusted Rand Index between the two solutions: {ari:.4f}  (1 = identical partitions)")
print("\nAgreement matrix (rows: K-Means, columns: Hierarchical):")
display(pd.crosstab(cust["Cluster"], cust["HCluster"]))
```

	Method	Clusters	Silhouette Score
0	K-Means	4	0.2291
1	Hierarchical (Ward)	4	0.1958

```
Adjusted Rand Index between the two solutions: 0.4585  (1 = identical partitions)

Agreement matrix (rows: K-Means, columns: Hierarchical):
```

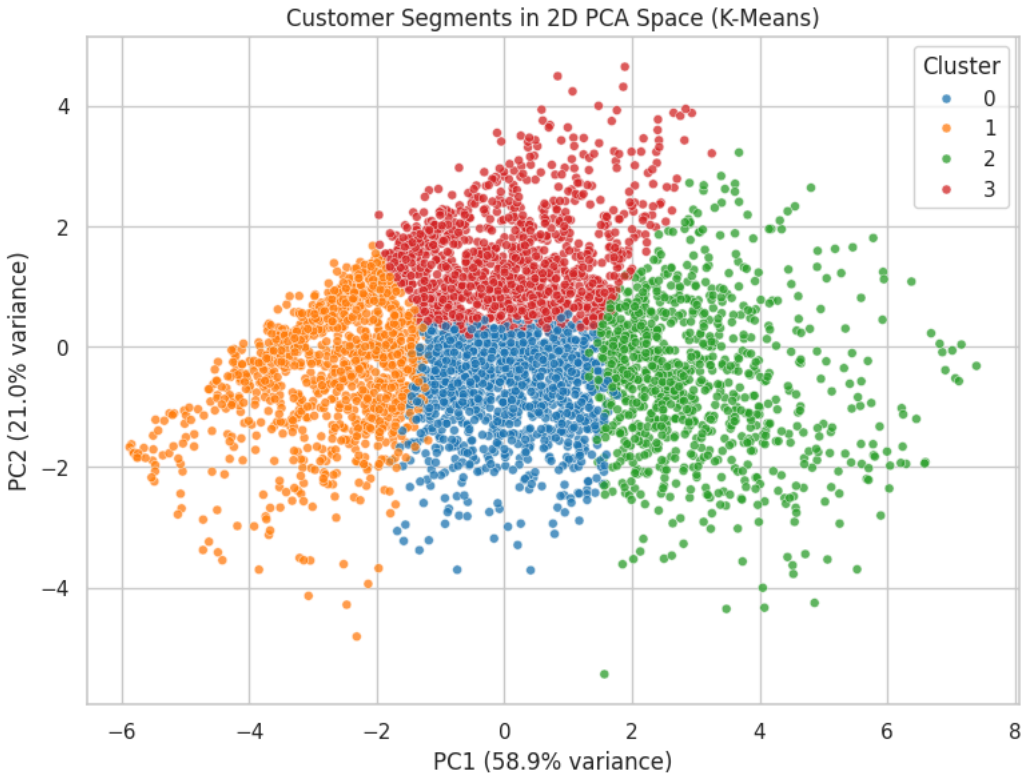
HCluster	0	1	2	3
Cluster				
0	35	64	812	221
1	801	0	261	1
2	0	775	19	139
3	347	3	39	821

```
pca = PCA(n_components=2, random_state=RANDOM_STATE)
pcs = pca.fit_transform(X)
print("Explained variance ratio:", pca.explained_variance_ratio_.round(3),
      "| Total:", pca.explained_variance_ratio_.sum().round(3))

plt.figure(figsize=(9, 6.5))
sns.scatterplot(x=pcs[:, 0], y=pcs[:, 1], hue=cust["Cluster"], palette="tab10", s=25, alpha=0.75)
plt.title("Customer Segments in 2D PCA Space (K-Means)")
plt.xlabel(f"PC1 ({pca.explained_variance_ratio_[0]:.1%} variance)")
plt.ylabel(f"PC2 ({pca.explained_variance_ratio_[1]:.1%} variance)")
plt.legend(title="Cluster"); plt.show()
```

```
loadings = pd.DataFrame(pca.components_.T, index=features, columns=["PC1", "PC2"])
display(loadings.round(3))
```

```
Explained variance ratio: [0.589 0.21 ] | Total: 0.799
```



	PC1	PC2
Recency	-0.293	0.422
Frequency	0.363	-0.332
Monetary	0.437	0.060
AvgTransactionValue	0.301	0.509
TotalQuantity	0.440	0.107
AvgQuantityPerOrder	0.306	0.495
PurchaseFrequency	0.287	-0.438
UniqueProducts	0.361	-0.023

```
profile = cust.groupby("Cluster")[features].mean()
profile.insert(0, "Customers", cust["Cluster"].value_counts().sort_index())
profile.insert(1, "Share %", (profile["Customers"] / len(cust) * 100))
display(profile.round(2))

# Heatmap of relative cluster means (z-score across clusters)
rel = (profile[features] - profile[features].mean()) / profile[features].std()
plt.figure(figsize=(10, 3.5 + 0.4 * K))
sns.heatmap(rel, annot=True, fmt=".2f", cmap="RdYlGn", center=0)
plt.title("Cluster profile (relative to other clusters)"); plt.show()

# Rule-based naming of segments
prof = cust.groupby("Cluster")[features].mean()
names, remaining = {}, list(prof.index)
hv = prof["Monetary"].idxmax();  names[hv] = "Champions (High-Value)"; remaining.remove(hv)
if remaining:
    lost = prof.loc[remaining, "Recency"].idxmax(); names[lost] = "Lost / Dormant"; remaining.remove(lost)
if remaining:
    new = prof.loc[remaining, "Frequency"].idxmin(); names[new] = "New / Low-Engagement"; remaining.remove(new)
for c in remaining:
    names[c] = "Loyal / Regular" if len(remaining) == 1 else f"Loyal / Regular (C{c})"
cust["Segment"] = cust["Cluster"].map(names)
HIGH_VALUE_CLUSTER = hv
print("Segment names:", names)
print(f"High-value cluster selected: {hv} -> '{names[hv]}' (highest mean Monetary value)")
```

Cluster	Customers	Share %	Recency	Frequency	Monetary	AvgTransactionValue	TotalQuantity	AvgQuantityPerOrder	PurchaseFrequer
0	1132	26.09	32.74	3.37	755.99	240.11	413.58	139.11	0
1	1063	24.50	172.95	1.45	217.22	160.93	101.54	76.07	0
2	933	21.51	20.82	11.61	6885.16	636.22	3978.64	371.17	1
3	1210	27.89	133.13	1.93	1137.75	640.73	713.90	421.94	0



Segment names: {np.int32(2): 'Champions (High-Value)', np.int32(1): 'Lost / Dormant', np.int32(3): 'New / Low-Engagement', 0: 'L

```
pred_features = [f for f in features if f not in ("Monetary", "AvgTransactionValue")]
y = (cust["Cluster"] == HIGH_VALUE_CLUSTER).astype(int)
X_sup = logged[pred_features]          # log-transformed, capped features (scaling happens inside the pipeline)

print(f"High-value customers: {y.sum()} ({y.mean():.1%}) | Others: {(1 - y).sum()}")

X_train, X_test, y_train, y_test = train_test_split(
    X_sup, y, test_size=0.25, stratify=y, random_state=RANDOM_STATE)

models = {
    "Random Forest": RandomForestClassifier(n_estimators=300, class_weight="balanced",
                                            random_state=RANDOM_STATE, n_jobs=-1),
    "SVM (RBF)": make_pipeline(StandardScaler(),
                               SVC(kernel="rbf", probability=True, class_weight="balanced",
                                   random_state=RANDOM_STATE)),
}

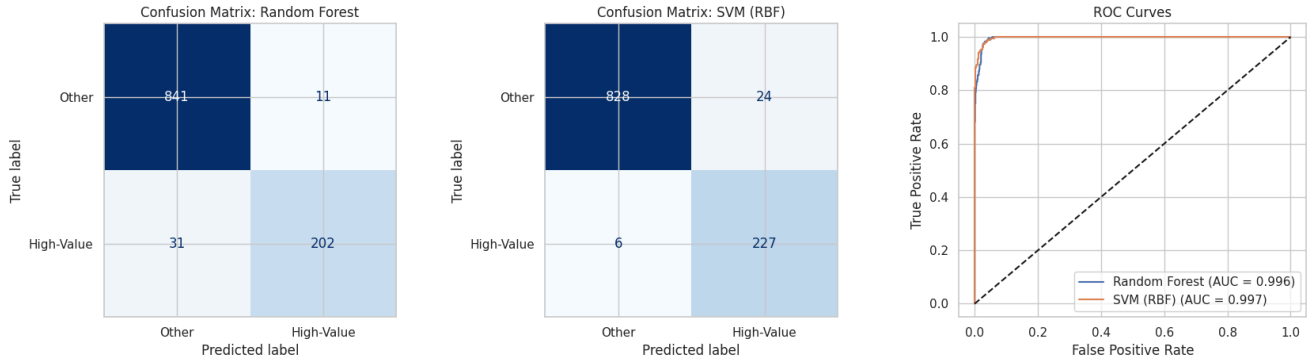
results, preds, probas = [], {}, {}
for name, model in models.items():
    model.fit(X_train, y_train)
    p = model.predict(X_test); pr = model.predict_proba(X_test)[:, 1]
    preds[name], probas[name] = p, pr
    cv = cross_val_score(model, X_sup, y, cv=StratifiedKFold(5, shuffle=True, random_state=RANDOM_STATE),
                         scoring="f1").mean()
    results.append({"Model": name,
                    "Accuracy": accuracy_score(y_test, p),
                    "Precision": precision_score(y_test, p),
                    "Recall": recall_score(y_test, p),
                    "F1-Score": f1_score(y_test, p),
                    "ROC-AUC": roc_auc_score(y_test, pr),
                    "5-fold CV F1": cv})
```

```
results_df = pd.DataFrame(results).set_index("Model")
display(results_df.round(4))
```

```
High-value customers: 933 (21.5%) | Others: 3405
```

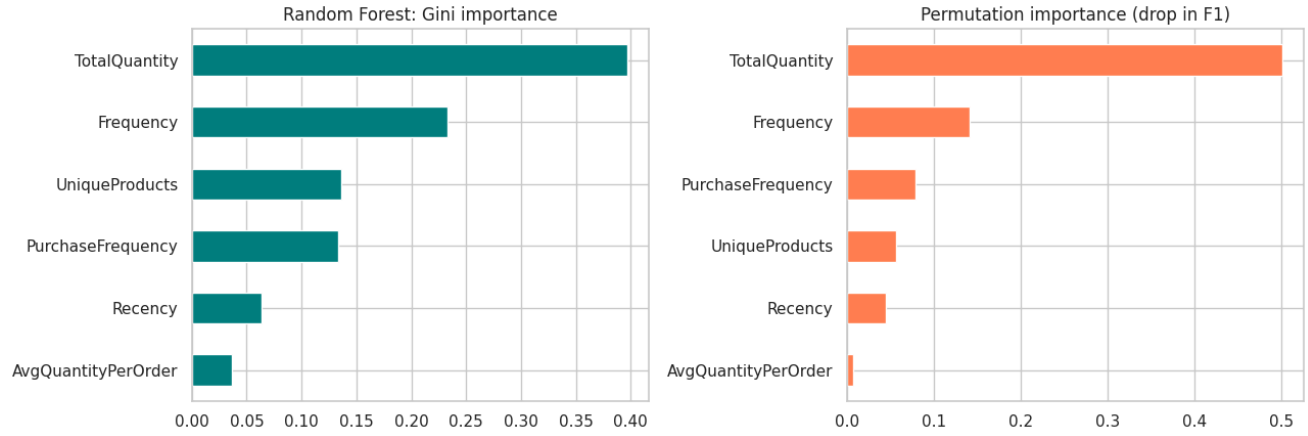
Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC	5-fold CV F1
Random Forest	0.9613	0.9484	0.8670	0.9058	0.9961	0.9448
SVM (RBF)	0.9724	0.9044	0.9742	0.9380	0.9974	0.9408

```
fig, axes = plt.subplots(1, 3, figsize=(17, 4.8))
for ax, name in zip(axes[:2], models):
    ConfusionMatrixDisplay(confusion_matrix(y_test, preds[name]),
                           display_labels=["Other", "High-Value"]).plot(ax=ax, cmap="Blues", colorbar=False)
    ax.set_title(f"Confusion Matrix: {name}")
for name in models:
    fpr, tpr, _ = roc_curve(y_test, probas[name])
    axes[2].plot(fpr, tpr, label=f"{name} (AUC = {roc_auc_score(y_test, probas[name]):.3f})")
axes[2].plot([0, 1], [0, 1], "k--"); axes[2].set_title("ROC Curves")
axes[2].set_xlabel("False Positive Rate"); axes[2].set_ylabel("True Positive Rate"); axes[2].legend()
plt.tight_layout(); plt.show()
```




```
rf = models["Random Forest"]
imp = pd.Series(rf.feature_importances_, index=pred_features).sort_values()
perm = permutation_importance(rf, X_test, y_test, n_repeats=15, scoring="f1",
                              random_state=RANDOM_STATE, n_jobs=-1)
perm_s = pd.Series(perm.importances_mean, index=pred_features).sort_values()

fig, ax = plt.subplots(1, 2, figsize=(13, 4.5))
imp.plot.barh(ax=ax[0], color="teal");   ax[0].set_title("Random Forest: Gini importance")
perm_s.plot.barh(ax=ax[1], color="coral"); ax[1].set_title("Permutation importance (drop in F1)")
plt.tight_layout(); plt.show()
```




```
plot_df = pd.DataFrame({
    "CustomerID": cust.index,
    "Segment": cust["Segment"],
    "Recency (days)": cust["Recency"],
    "Frequency (log)": np.log1p(cust["Frequency"]),
    "Monetary (log)": np.log1p(cust["Monetary"]),
    "Monetary": cust["Monetary"].round(2),
    "Frequency": cust["Frequency"],
})
fig = px.scatter_3d(plot_df, x="Recency (days)", y="Frequency (log)", z="Monetary (log)",
                    color="Segment", hover_data=["CustomerID", "Frequency", "Monetary"],
                    opacity=0.75, title="Customer Segments: Recency vs Frequency vs Monetary")
fig.update_traces(marker=dict(size=3))
fig.update_layout(height=700, legend_title_text="Segment")
fig.show()

# Optional: 3D PCA view
pcs3 = PCA(n_components=3, random_state=RANDOM_STATE).fit_transform(X)
fig2 = px.scatter_3d(x=pcs3[:, 0], y=pcs3[:, 1], z=pcs3[:, 2], color=cust["Segment"],
                     labels={"x": "PC1", "y": "PC2", "z": "PC3", "color": "Segment"},
                     opacity=0.75, title="Customer Segments in 3D PCA Space")
fig2.update_traces(marker=dict(size=3)); fig2.update_layout(height=700)
fig2.show()
```

Customer Segments: Recency vs Frequency vs Monetary

Customer Segments in 3D PCA Space

```
playbook = {
    "Champions": ("Reward and retain",
                  ["VIP / loyalty tier with early access to new products",
                   "Personalised bundles and bulk-order discounts",
                   "Referral and ambassador programmes; avoid heavy discounting (they already buy)"]),
    "Loyal": ("Grow basket size",
              ["Cross-sell and upsell using purchase history",
               "Subscription or replenishment reminders",
               "Points-based rewards that push them toward the Champion tier"]),
    "New": ("Nurture and convert",
            ["Welcome series and onboarding emails",
             "Second-purchase incentive (time-limited coupon)",
             "Product education and best-seller recommendations"]),
    "Lost": ("Win back at controlled cost",
             ["Win-back campaign with a strong, expiring offer",
              "Short survey to learn why they left",
              "Suppress from expensive channels if there is no response"]),
}
summary = []
for c, seg in names.items():
    key = next(k for k in playbook if seg.startswith(k))
    goal, actions = playbook[key]
    row = prof.loc[c]
    print(f"\n=== {seg} (Cluster {c}, {int((cust['Cluster'] == c).sum())} customers) ===")
    print(f"Profile : Recency {row['Recency']:.0f} days | Frequency {row['Frequency']:.1f} orders | "
          f"Spend {row['Monetary']:.0f} | Avg order {row['AvgTransactionValue']:.0f}")
    print(f"Goal    : {goal}")
    for a in actions: print("  -", a)

rev = cust.groupby("Segment")["Monetary"].agg(["count", "sum"]).rename(columns={"count": "Customers", "sum": "Revenue"})
rev["Revenue share %"] = rev["Revenue"] / rev["Revenue"].sum() * 100
display(rev.sort_values("Revenue", ascending=False).round(1))
```

```
=== Champions (High-Value) (Cluster 2, 933 customers) ===
Profile : Recency 21 days | Frequency 11.6 orders | Spend 6885 | Avg order 636
Goal    : Reward and retain
  - VIP / loyalty tier with early access to new products
  - Personalised bundles and bulk-order discounts
  - Referral and ambassador programmes; avoid heavy discounting (they already buy)

=== Lost / Dormant (Cluster 1, 1063 customers) ===
```